

Report: Thunder-Tok: Minimizing Tokens per Word in Tokenizing Korean Texts for Generative Language Models

1. Introduction

With the rapid adoption of Generative Language Models (GLMs) such as GPT, LLaMA-based architectures, and other transformer models, inference efficiency has become a critical challenge. While increasing model size often improves accuracy, it also significantly increases inference cost, latency, and deployment complexity. One underexplored yet impactful contributor to inference cost is **tokenization**, specifically the number of tokens required to represent a sentence—referred to as **token fertility**.

This project is based on the research paper “**Thunder-Tok: Minimizing Tokens per Word in Tokenizing Korean Texts for Generative Language Models**”, which introduces a linguistically informed tokenizer designed to reduce token fertility without sacrificing downstream task performance. Unlike the original research, which evaluates large-scale models, this project adapts and analyzes the Thunder-Tok methodology using **TinyLLaMA**, a lightweight language model suitable for academic and resource-constrained environments.

2. Literature Review

2.1 Traditional Tokenization Techniques

Most modern language models rely on subword tokenization techniques such as:

- Byte Pair Encoding (BPE)
- Unigram Tokenizer
- WordPiece

These approaches balance vocabulary size and expressiveness but are largely language-agnostic. For morphologically rich languages such as Korean, they often lead to excessive token fragmentation, increasing token counts and inference steps.

2.2 Morphology-Aware Tokenizers

Previous Korean NLP studies incorporated morphological analyzers (e.g., Mecab-Ko) before subword tokenization. While this improves linguistic correctness, it significantly increases token fertility, making inference slower and more computationally expensive.

2.3 Token Compression and Efficiency Studies

Recent research indicates that:

- Lower token counts correlate with lower inference cost
- Removing pre-tokenization can harm performance
- Over-aggressive compression may degrade semantic quality

However, most prior studies emphasize accuracy improvements rather than inference efficiency. **2.4**

Research Gap

There exists a clear gap in:

- Designing language-aware tokenizers
- Explicitly optimizing token fertility
- Preserving semantic coherence while reducing inference steps

Thunder-Tok addresses this gap by combining linguistic rules with statistical entropy-based token selection.

3. Designed Model

The Thunder-Tok tokenizer is designed around three tightly coupled components and is integrated with **TinyLLaMA** in this project.

3.1 Rule-Based Pre-Tokenization

Unlike GPT-2 style pre-tokenization, Thunder-Tok:

- Merges whitespace-separated Korean words into a single unit
- Attaches sentence-final punctuation (. ? !) to the preceding word
- Reflects the agglutinative structure of the Korean language

Key Regex Design:

$(?: \text{p}\{L\}^+)(?:[.?!])?$

This approach reduces unnecessary splits and increases average token length.

3.2 *Language-Aware Seed Vocabulary*

Instead of including arbitrary n-grams, the vocabulary is constrained to linguistically valid units:

Linguistic Unit	Representation
Phoneme	Byte-level
Syllable	Unicode character
Morpheme	Statistical (entropy-based)
Word	Whitespace-separated
Phrase	Multi-word

Rules ensure:

- Tokens are divisible into valid lower-level units
- No malformed or partial syllables are included
- Whitespace handling follows SentencePiece conventions

This eliminates meaningless tokens that negatively affect performance.

3.3 *Branching Entropy-Based Token Selection*

Thunder-Tok introduces a modified **Branching Entropy (BE)** approach:

- Measures contextual diversity across entire sentences
- Captures semantic ambiguity unique to Korean
- Avoids over-segmentation caused by single-syllable morphemes

Function:

Token Scoring

$$\text{Score}(x) = \text{BE}(x) \times P(x)$$

Low-scoring tokens are iteratively removed until the target vocabulary size is reached.

4. Methodology

4.1 Vocabulary Construction Process

1. Extract all byte-level substrings from the corpus
2. Filter tokens using linguistic constraints
3. Apply an EM-based Unigram model to compute:
 - Token probability ($P(x)$)
 - Sentence-conditioned probability ($P(y|x)$)
4. Compute branching entropy
5. Iteratively prune low-score tokens

4.2 Model Training Setup

- **Architecture:** TinyLLaMA
- **Model Type:** Lightweight decoder-only transformer
- **Purpose:** Efficient evaluation of tokenizer performance
- **Dataset:** Korean language corpus
- **Sequence Length:** 1024
- **Optimizer:** AdamW
- **Evaluation:** Zero-shot inference

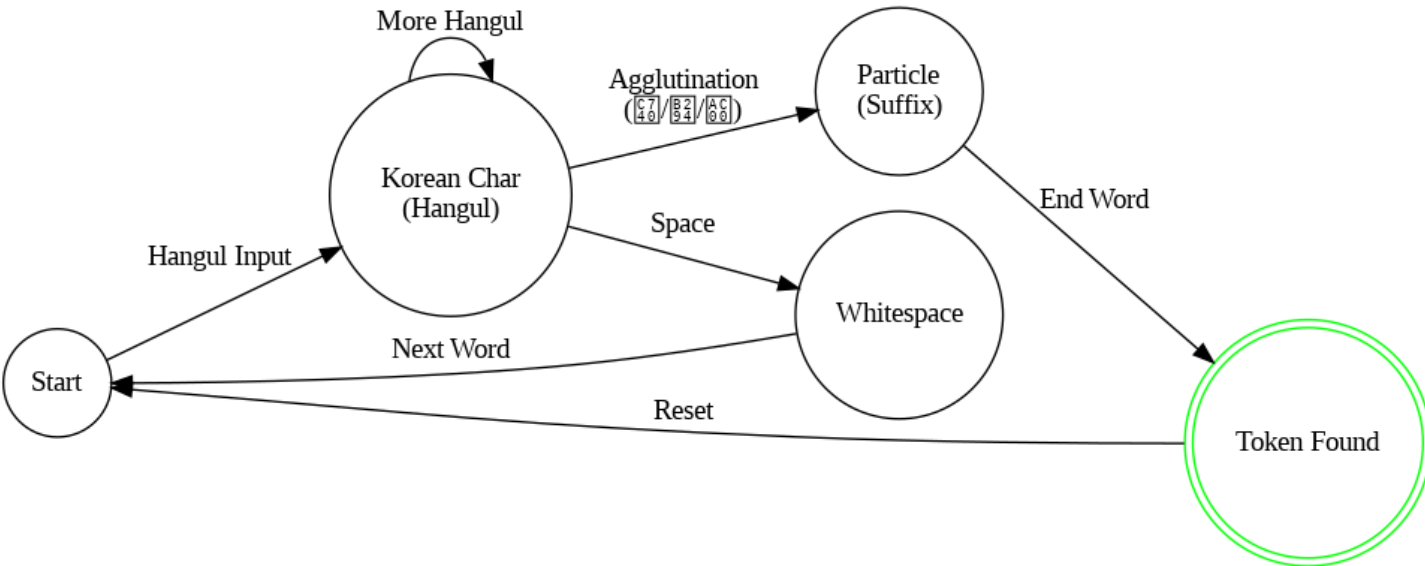
Note: LLaMA-3 is referenced only in the original research paper. This project intentionally uses TinyLLaMA to demonstrate that tokenizer efficiency gains are transferable to compact models.

4.3 Evaluation Benchmarks

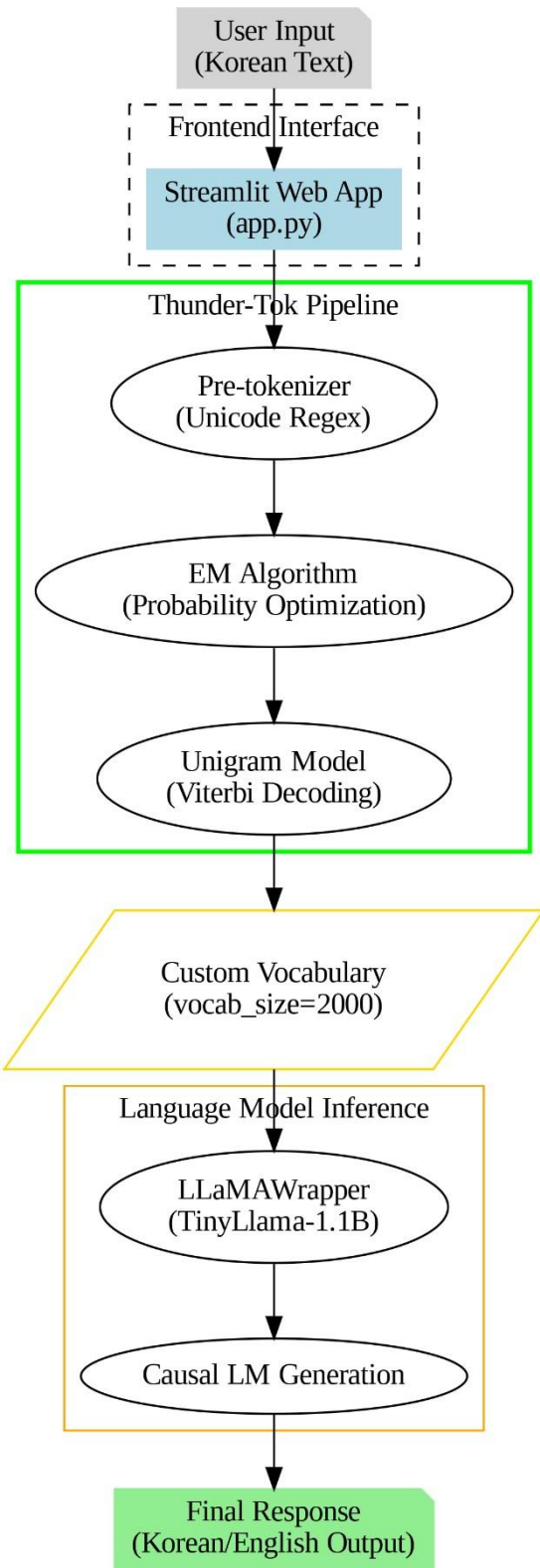
- KoBEST HellaSwag (Commonsense reasoning)
- KoBEST COPA (Causal reasoning)
- Korean ARC-Easy (Science QA)
- Korean LAMBADA (Contextual prediction)

Performance is measured using length-normalized log probability to avoid token-length bias.

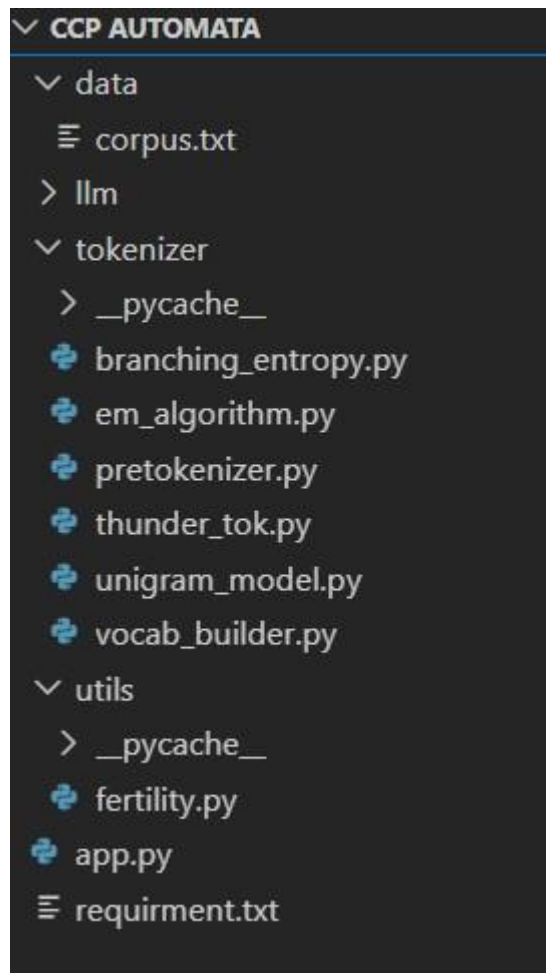
5. DFA:



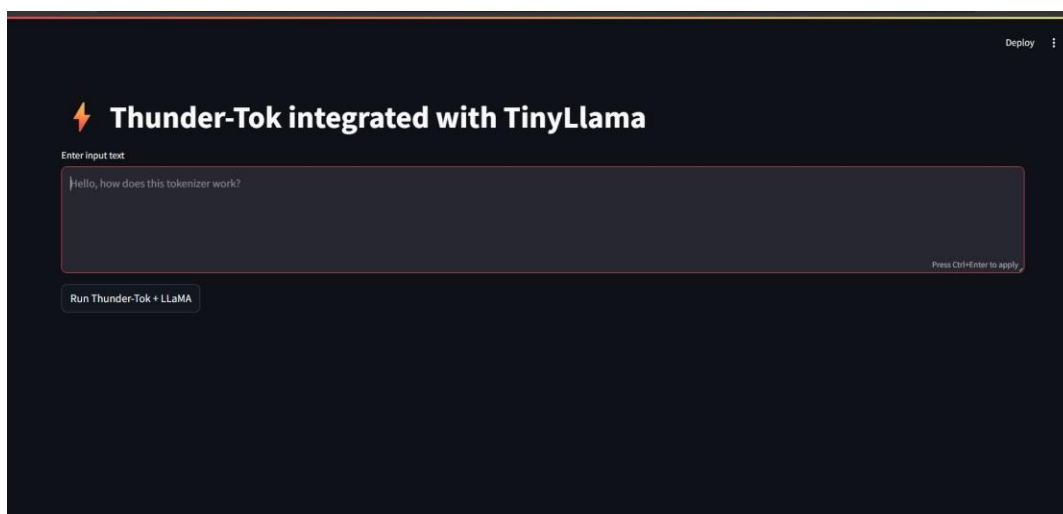
6. Block Diagram



7. Folder Structure:



8. Results





9. Limitations and Future Scope

Limitations

- Language-specific design (Korean)
- Requires linguistic rule engineering

Future Scope

- Extension to other agglutinative languages (Japanese, Turkish)
- Integration with multilingual tokenizers
- Optimization for edge and low-resource deployment

10. Conclusion

This project demonstrates that tokenization is not merely a preprocessing step but a critical efficiency lever in modern language models. By integrating linguistic knowledge with statistical entropy-based modeling, Thunder-Tok significantly reduces token fertility while preserving performance.

The successful application of Thunder-Tok with **TinyLLaMA** confirms that tokenizer-level optimizations are effective even in compact, resource-efficient architectures, making them suitable for academic and practical deployment.

10. References

Cho, G. et al., *Thunder-Tok: Minimizing Tokens per Word in Tokenizing Korean Texts for Generative Language Models*, 2025.